

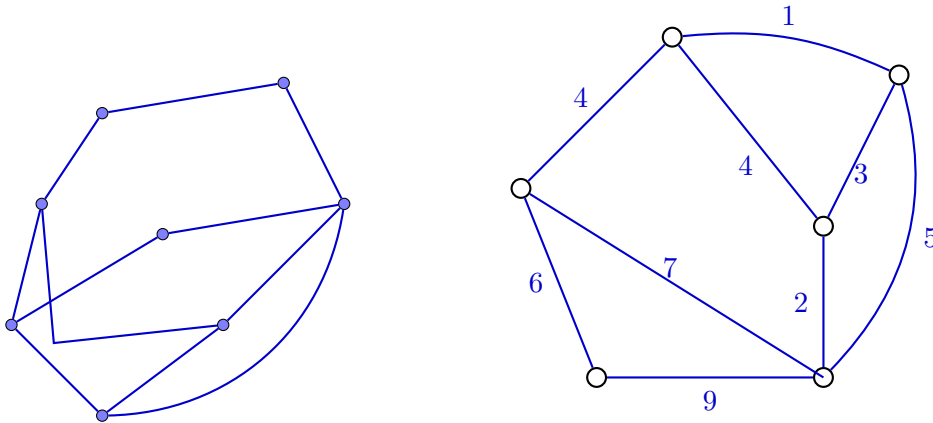
Greedy Algorithm: Minimum Spanning Trees

Introduction

Given an undirected graph $G(V, E)$, a **spanning tree** is a subgraph that is a tree and which includes all the vertices of G .

- Spanning tree should include all vertices.
- Connected subgraph without cycle.
- Spanning tree has $V - 1$ edges.

Now what is Minimum spanning tree: Minimum Spanning tree of the graph $G(V, E)$ is a spanning tree of minimum weight/length/cost in the graph. We are given a length function $l : E \rightarrow R^+$. Assume we have non-negative weight.



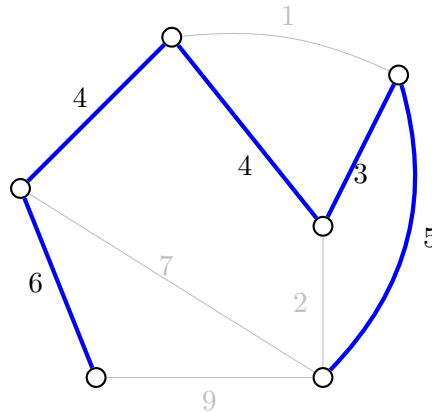
Think of the problem this way, there are certain cities and we need to connect them by wire. So we have plenty of options to connect cities by wire. Some of the options are not possible due to some infeasibility reason. So this is the graph so connect cities by wire.

I am also told how much the length of wire I need to connect them. Suppose the lengths of the wire needed are as shown in the figure. So we need to connect the cities which means I need to create spanning trees and I am not interested in any spanning tree. I am interested in a spanning tree for which the length of the wire used is as small as possible. So that is what we mean by minimum spanning tree, a spanning tree of min length; so what is the length of a spanning tree now it is mostly but the sum of the lengths of the edges in the tree.

Let the spanning tree: it equals the sum of the lengths of edges in the tree. Formally define that the length of the tree.

Analyzing a Spanning Tree

So I might now decide to pick some edges. Suppose I pick... Let us pick the edges as shown.



What is the length of the tree?

$$6 + 4 + 4 + 3 + 5 = 22$$

There might be other spanning trees which are smaller than 22. We are interested in finding the spanning tree of smallest weight. There is no real notion of the root in the tree here.

One way you convince yourself that this is the smallest possible tree is by seeing that for instance, if I include the edge of length 1 then what is going to happen? Will it still be a tree? No we will have a cycle, so that is one property of a spanning tree always.

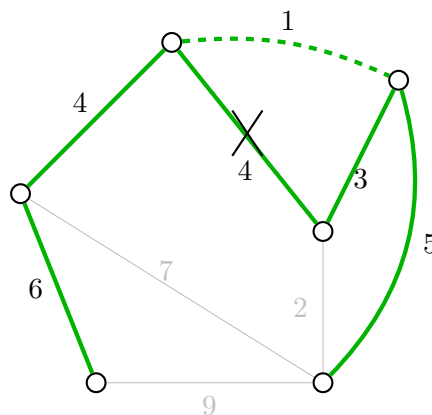
Suppose you have this spanning tree. Spanning tree means connected subgraph between every two vertices there is a path already. Now we do add one more edge now, It will form a cycle, because between u and v there is already a path. So adding the edge between u and v (shown dotted) it will form a cycle it means it is no longer a spanning tree.

Now suppose in this cycle each has a certain length. Suppose the lengths are as shown, include edge of length 1. Now what can I do? I can include the edge with length 1 and drop the max length in the cycle.

Now question is will that continue to be a tree? Why would it remain connected? To add an edge in the tree that addition will clearly form a cycle and if it forms a cycle I can drop any edge of longer length min cost of the tree.

Iterative Optimization Example

For instance if I add the edge of length 1 Then I would form a cycle.



Now I can drop any edge of cycle and it still remain connected. Now which edge I would like to drop? Edge with cost 4.

Now the cost of the tree becomes:

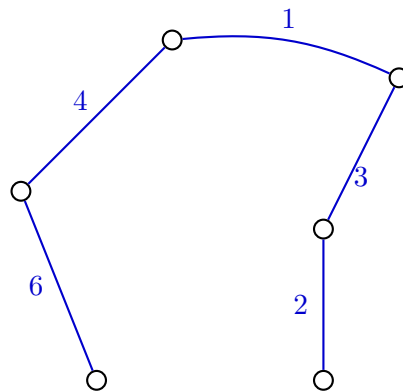
$$22 - 3 = 19 \quad (\text{replacing 4 with 1 decreases cost by 3})$$

So I can repeat the process and see if there is something else that I can drop. Now we can drop 5 and include 2.

Now the weight becomes:

$$19 - 3 = 16 \quad (\text{replacing 5 with 2 decreases cost by 3})$$

So the tree that I have at this point is:



Would it make sense to include other edges and try out by doing repeat the process? No. So it does not make sense to include other edge. And this is the minimum I should say. This is not the reason that it is min spanning tree. There are other different arguments to prove that this is the minimum spanning tree.

So this could be treated as an algorithm to compute the min spanning tree. How would it work? You start with a tree, then try to include an edge look at the cycle gets formed so you can drop an edge of longer length from that cycle, therefore reduce the cost, so you can keep doing this procedure till you reduce the cost, and stop when you can't reduce any more.

But we will not look at this algorithm, or analyze this algorithm because it will be fairly expensive in terms of running time. So we are going to look at different algorithm and it is called **Kruskal algorithm** for computing the MST.

Kruskal's Algorithm

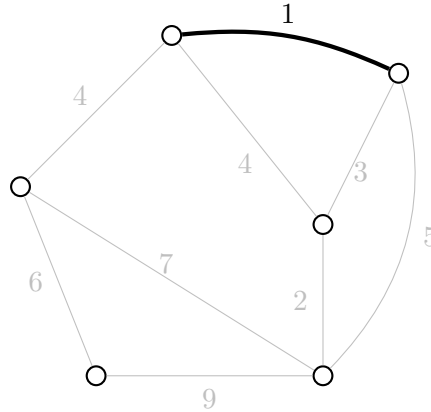
Let me illustrate on the same graph that we had before.

So the algorithm says the following: - Take the minimum edge of the graph.

The algorithm is called **greedy**. So what do you think greedy means here. *Make the best possible choice without thinking of the future.* That's the key thing.

Here our objective is to minimize the length. So what is the best thing to do at the 1st step? Pick/take the edge with smallest length, and it is included in the min spanning tree. We are constructing the tree edge by edge.

So we have included the edge of length 1.



Now which is the next best choice to make? 2. Or what you think that it should remain connected then you are looking at the future, dont do that, be greedy. Just make the best choice.

Next best is the edge 2. Now what does greedy say, which is the next best choice? Edge 3. Right, so 3 is also included.

Next there are two 4s. Pick one. Now if we pick other edge 4 it will form a cycle. So if we include this it will not be tree any more. So we will not include this edge. Inclusion of an edge form a cycle I dont include that edge.

Next try to include 5 but that will form a cycle we wont include that edge. What next? 6.

Now we can stop at this point as well we have covered all the vertices and it is the spanning tree any other edge that I try to include will form a cycle because this is a spanning tree.

We have not yet argued that this is the best possible tree. Tell an argument for why this is the best possible... but this is the same solution that we have seen obtained earlier.

Algorithm Steps

Let us quickly write down what the algorithm is.

Kruskal algorithm for MST

1. Sort edges in increasing order of length: $l(e_1) \leq l(e_2) \cdots \leq l(e_m)$
2. Let $T = \emptyset$ (set of edges)
3. for $i = 1$ to m do
 - if $T \cup \{e_i\}$ is a tree
 - $T = T \cup \{e_i\}$
4. Return T

How do you check that the edge that you are trying to include forms a cycle or not? This is very important step and it will dictate the running time of the algorithm. So if the checking of cycle takes $m \log m$ time then the algorithm would be an $O(m \log m)$ algorithm.

Proof of Correctness

Before that we have to argue why this is the MST? Why there would not be any other spanning tree smaller than this? So we will look at the tree edges picked by Kruskal algorithm.

Let me name the edges picked by Kruskal algorithm:

$$g_1 < g_2 < g_3 < \cdots < g_{v-1}$$

Suppose some how picked the best spanning tree by searching the graph in brute force way and let the name of the edges of the MST be (optimum tree):

$$f_1 < f_2 < f_3 < \dots < f_{v-1}$$

For simplicity assume all edge lengths are distinct. These two sets of edges need not be the same. What we will argue is that they are indeed the same. If they are the same then what we have found out by the Kruskal algorithm is the best tree.

Proof is by **contradiction**.

Suppose these sets differ & the 1st place where they differ is i .

$$g_1 < g_2 < g_3 \dots g_{i-1} < g_i \dots$$

$$f_1 < f_2 < f_3 \dots f_{i-1} < f_i \dots$$

We start from the left and see which edge differ:

$$g_1 = f_1, \quad g_2 = f_2, \quad \dots \quad g_{i-1} = f_{i-1} \quad \text{but} \quad g_i \neq f_i$$

We will argue that this can not happen. There are two possibilities now:

i) $l(g_i) < l(f_i)$

ii) $l(g_i) > l(f_i)$

Case 1: $l(g_i) < l(f_i)$

Assume $l(g_i) < l(f_i)$. All the edges after f_i are larger than f_i . So g_i can not be there in $f_i \dots f_{v-1}$ because all edges are of distinct length. And it can not be there in $f_1 \dots f_{i-1}$ either because $f_1 \dots f_{i-1}$ are same as $g_1 \dots g_{i-1}$ respectively and they are all distinct.

That means g_i is not there in $f_1 \dots f_{v-1}$. i.e., g_i is not in the optimum tree.

So we can add g_i in the optimum. * Add g_i to optimum. It could form a cycle. Let the cycle be C .

Can it happen that g_i is of largest length in the cycle C ? If g_i is largest length of all the edges in C then it would have formed cycle in the Algorithm formed tree, because $f_1 \dots f_{i-1}$ are identical to $g_1 \dots g_{i-1}$ so can only be longer in the cycle. One form of all other edges in the cycle f_1 to f_{i-1} are identical to g_1 to g_{i-1} .

So if g_i has formed a cycle in Opt then it would have formed cycle in the tree formed by Kruskal algorithm.

If any edge in the cycle C is larger than g_i then we are done. Why? We will include g_i and remove the edge of lesser length and cost of the optimum tree would reduce, and get a contradiction.

Case 2: $l(g_i) > l(f_i)$

f_i is distinct from $g_i \dots g_{v-1}$. It also means f_i is distinct from $g_1 \dots g_{i-1}$.

Now question is why didn't we pick f_i in the Kruskal algorithm? It could have formed a cycle.

$$f_i \cup \{g_1, \dots, g_{i-1}\} \text{ contain cycle}$$

$$\implies f_i \cup \{f_1, \dots, f_{i-1}\} \text{ also contains cycle}$$

$$\implies \text{Opt also contains a cycle}$$

But that is not possible, because it is a tree. So thats the contradiction.

So we can say that there is no place where these two differ. These two are one and the same. Kruskal algorithm produce MST.

Now is the MST unique?

Yes if edge lengths are distinct.

No otherwise.